

Mini proyecto gráfico con Maven, Processing y VS Code

Objetivo

Crear una aplicación Java con Maven que permita graficar una función matemática elegida desde un menú.

El proyecto permitirá mostrar:

- uso de Maven,
- incorporación de dependencias,
- uso de una biblioteca gráfica externa,
- generación de un JAR ejecutable.

1) Crear el proyecto desde VS Code

Abrir VS Code.

Presionar:

Ctrl + Shift + P

Buscar:

Maven: Create Maven Project

Seleccionar:

maven-archetype-quickstart

Completar:

```
groupId: com.ejemplo  
artifactId: mini-dibujo  
version: 1.0-SNAPSHOT  
package: com.ejemplo
```

2) Revisar la estructura creada

El proyecto tendrá una estructura similar a:

```
mini-dibujo
├── pom.xml
├── src
│   ├── main
│   │   └── java
│   │       ├── com
│   │       │   └── ejemplo
│   │       │       └── App.java
│   └── test
```

3) Reemplazar el pom.xml

Abrir `pom.xml` y reemplazarlo por:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://
maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.ejemplo</groupId>
  <artifactId>mini-dibujo</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.processing</groupId>
      <artifactId>core</artifactId>
      <version>4.5.5</version>
    </dependency>
  </dependencies>
```

```

<build>
  <plugins>

    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.13.0</version>
    </plugin>

    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-shade-plugin</artifactId>
      <version>3.6.0</version>

      <executions>
        <execution>
          <phase>package</phase>

          <goals>
            <goal>shade</goal>
          </goals>

          <configuration>
            <transformers>
              <transformer
implementation="org.apache.maven.plugins.shade.resource.ManifestResourceTransformer">
                <mainClass>com.ejemplo.App</mainClass>
              </transformer>
            </transformers>

            <finalName>mini-dibujo</finalName>
          </configuration>
        </execution>
      </executions>
    </plugin>

  </plugins>
</build>

</project>

```

4) Reemplazar `App.java`

Abrir:

```
src/main/java/com/ejemplo/App.java
```

Reemplazar su contenido por:

```

package com.ejemplo;

import javax.swing.JOptionPane;
import processing.core.PApplet;

public class App extends PApplet {

    private static String funcionSeleccionada = "sen(x)";

    public void settings() {
        size(800, 600);
    }

    public void setup() {
        surface.setTitle("Graficador de funciones");
    }

    public void draw() {
        background(245);

        dibujarEjes();
        graficarFuncion();
        escribirTitulo();
    }

    public void dibujarEjes() {
        stroke(0);
        strokeWeight(2);

        line(50, height / 2, width - 50, height / 2);
        line(width / 2, 50, width / 2, height - 50);
    }

    public void graficarFuncion() {
        stroke(255, 0, 0);
        strokeWeight(3);
        noFill();

        beginShape();

        for (float xPixel = 50; xPixel <= width - 50; xPixel++) {

            double x = map(xPixel, 50, width - 50, -10, 10);
            double y = calcularY(x);

            if (!Double.isNaN(y) && !Double.isInfinite(y)) {
                float yPixel = map((float) y, -10, 10, height - 50, 50);
                vertex(xPixel, yPixel);
            }
        }
    }
}

```

```

        endShape();
    }

    public double calcularY(double x) {

        switch (funcionSeleccionada) {
            case "sen(x)":
                return Math.sin(x);

            case "cos(x)":
                return Math.cos(x);

            case "x2":
                return x * x;

            case "x3":
                return x * x * x;

            case "ln(x)":
                if (x > 0) {
                    return Math.log(x);
                } else {
                    return Double.NaN;
                }

            default:
                return Math.sin(x);
        }
    }

    public void escribirTitulo() {
        fill(0);
        textSize(24);
        text("Funcion: y = " + funcionSeleccionada, 260, 40);
    }

    public static void main(String[] args) {

        String[] opciones = {
            "sen(x)",
            "cos(x)",
            "x2",
            "x3",
            "ln(x)"
        };

        String seleccion = (String) JOptionPane.showInputDialog(
            null,
            "Seleccione la funcion a graficar:",
            "Menu de funciones",

```

```
        JOptionPane.QUESTION_MESSAGE,  
        null,  
        opciones,  
        opciones[0]  
    );  
  
    if (seleccion != null) {  
        funcionSeleccionada = seleccion;  
        PApplet.main("com.ejemplo.App");  
    }  
}  
}
```

5) Actualizar Maven en VS Code

Guardar los archivos.

Si VS Code muestra el aviso:

Maven project needs update

hacer clic en:

Update Project

También se puede usar:

Ctrl + Shift + P
Maven: Update Project

6) Compilar el proyecto

Abrir la terminal integrada:

Ctrl + Ñ

Ejecutar:

mvn clean compile

Maven descargará la dependencia de Processing y compilará el proyecto.

7) Ejecutar desde VS Code

Abrir `App.java`.

Ejecutar desde:

Run Java

Aparecerá primero un menú para elegir la función:

sen(x)
cos(x)
 x^2
 x^3
ln(x)

Luego se abrirá la ventana gráfica.

8) Generar el JAR ejecutable

En la terminal ejecutar:

mvn clean package

Se generará:

target/mini-dibujo.jar

9) Ejecutar fuera de VS Code

Desde la terminal:

java -jar target/mini-dibujo.jar

También se puede copiar `mini-dibujo.jar` a otra carpeta y ejecutarlo con:

java -jar mini-dibujo.jar

Explicación para los alumnos

Processing es una biblioteca externa que permite dibujar ventanas, líneas, textos, colores y formas.

La agregamos al proyecto mediante Maven:

```
<dependency>
  <groupId>org.processing</groupId>
  <artifactId>core</artifactId>
  <version>4.5.5</version>
</dependency>
```

Gracias a Maven no necesitamos descargar manualmente archivos `.jar`.

Además, usamos el plugin `maven-shade-plugin` para generar un JAR ejecutable que contiene también las dependencias necesarias.

En este ejemplo:

- Java aporta la lógica del programa.
- `Math` aporta las funciones matemáticas.
- Processing aporta la parte gráfica.
- Maven administra las dependencias y genera el JAR final.